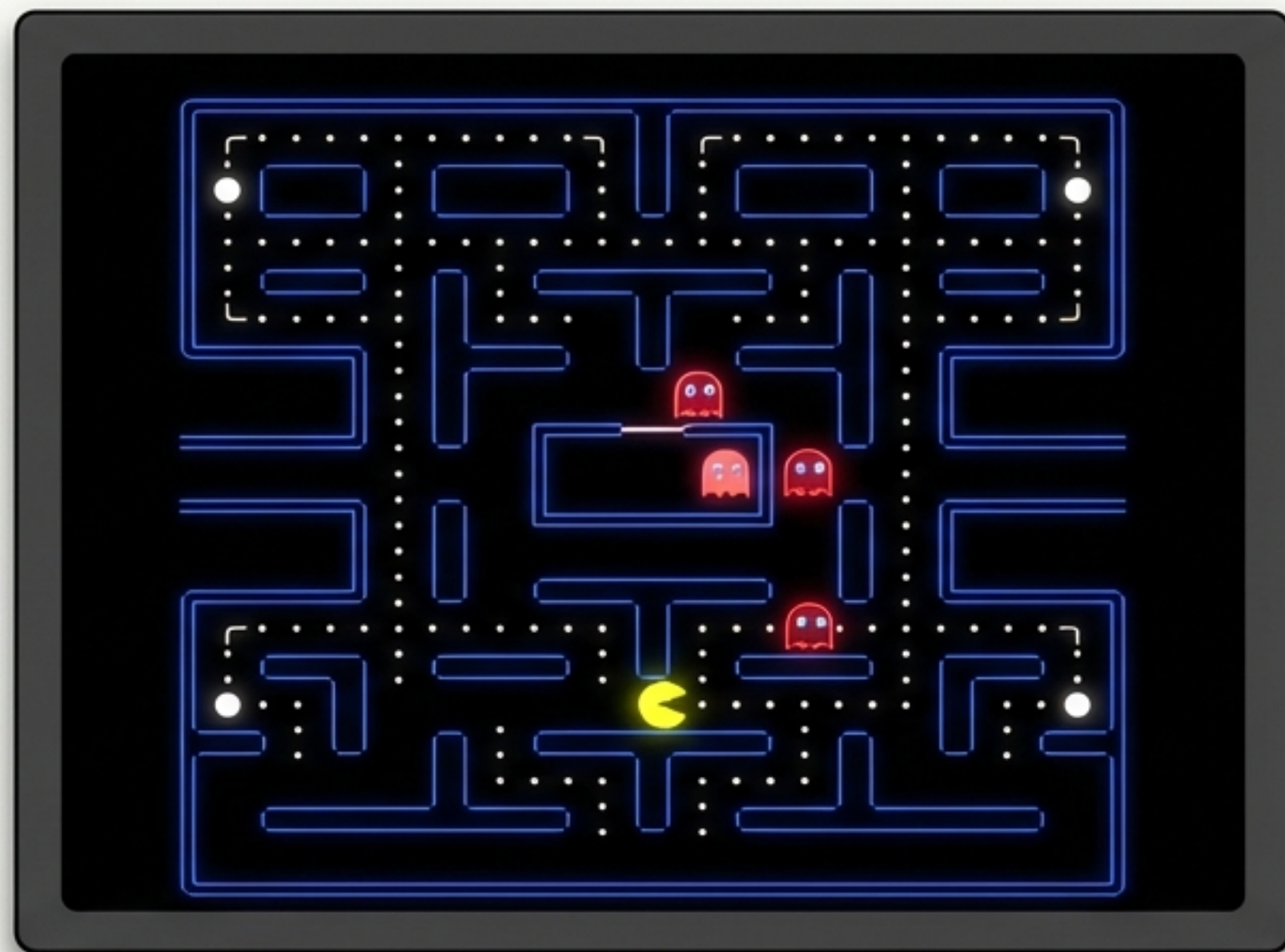




Pac-Man no Terminal: A Anatomia de um Jogo em Python

Uma análise profunda de como lógica, estrutura e criatividade dão vida a um clássico no console.

O Palco: Construindo o Labirinto a Partir de Texto

A fundação de todo o jogo reside em como definimos seu mundo. Em vez de complexos editores de nível, usamos uma única string multi-linha para "desenhar" o labirinto. Cada caractere tem um propósito: `\*` para os caminhos, e caracteres como `┌`, `|`, `┐` para as paredes.

```
MAPA_ORIGINAL = r"""
```

```
┌*****┐┌*****┐┌*****┐
│*  ┌───┐*││*  ┌───┐*││*  ┌───┐*│
│... (mapa continua, mas levemente esmaecido) ...
└*****┘└*****┘└*****┘
```

```
"""
```



Paredes do
Labirinto



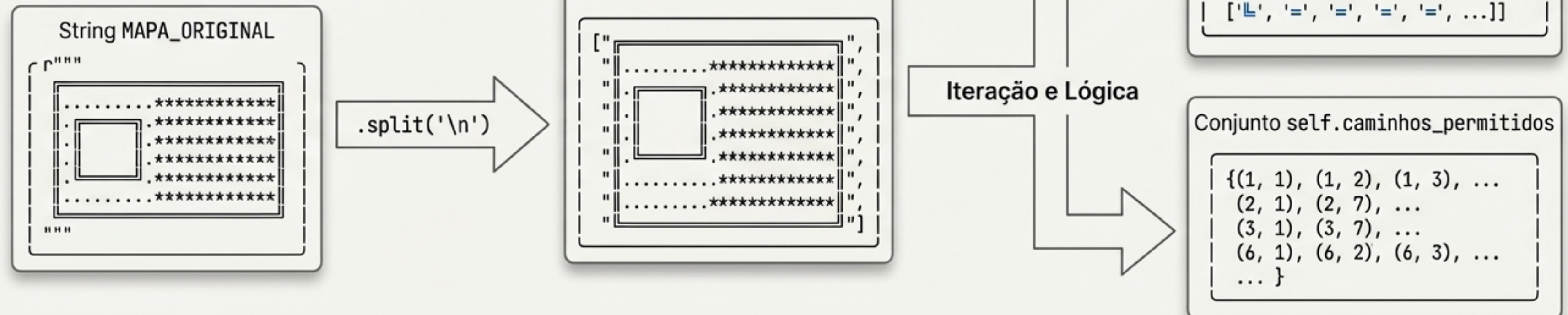
Caminho e
Pastilha



A Matriz: Transformando Texto em Lógica Interativa

No construtor da classe (`__init__`), o mapa em texto ganha vida. O código itera sobre a string, convertendo-a em uma lista de listas (`self.mapa`), que representa o grid do jogo. Simultaneamente, ele identifica todos os caminhos válidos (onde havia um `*`) e os armazena em um conjunto (`self.caminhos_permitidos`) para verificações de colisão rápidas e eficientes.

```
# Dentro de __init__(self, mapa_str):
self.caminhos_permitidos = set()
for y in range(self.altura):
    for x in range(len(self.mapa[y])):
        char = self.mapa[y][x]
        if char == '*':
            self.caminhos_permitidos.add((y, x))
            self.mapa[y][x] = PASTILHA
```



O Nascimento do Herói: Definindo o Protagonista

O nosso herói é definido por alguns atributos-chave na classe do jogo. `self.pacman\_pos` guarda suas coordenadas `[y, x]`. Suas diferentes aparências, dependendo da direção, são armazenadas em um dicionário `PACMAN\_SPRITES`, que mapeia uma tecla de direção ('w', 'a', 's', 'd') para um caractere Unicode.

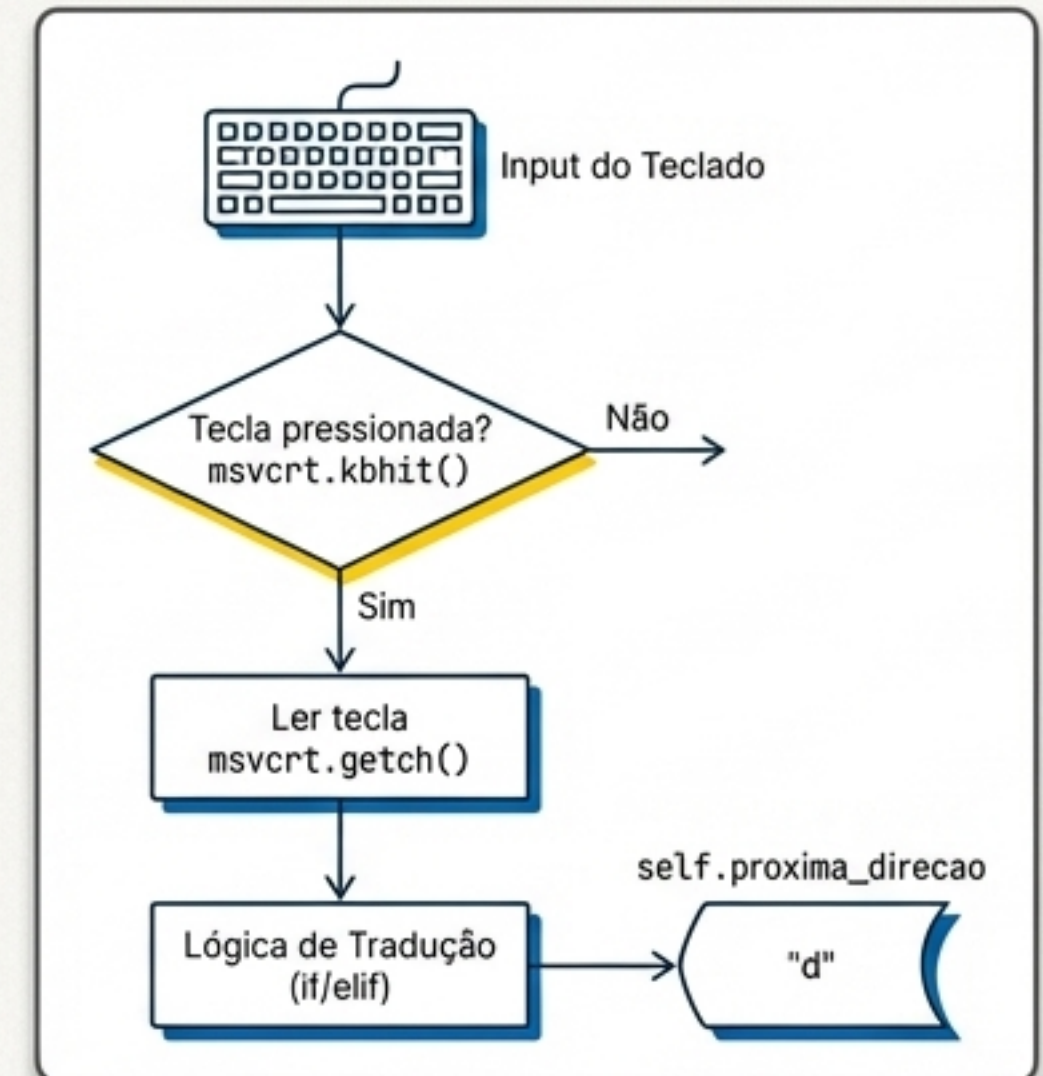
```
# Atributos do Pac-Man
self.pacman_pos = [1, 2] # Posição inicial
self.direcao_atual = 'd'
self.proxima_direcao = 'd'
```

```
# "Sprites" do Pac-Man
PACMAN_SPRITES = {
    'd': '👤', # Direita
    'a': '👤', # Esquerda
    'w': '👤', # Cima
    's': '👤' # Baixo
}
```


O Controle: Como o Jogo Ouve o Jogador em Tempo Real

Para que o jogo continue rodando enquanto espera por um comando, não podemos usar um `input()` padrão. A solução é a função `ler_input`, que utiliza a biblioteca `msvcrt`. A função `msvcrt.kbhit()` verifica se uma tecla foi pressionada sem travar a execução. Se foi, `msvcrt.getch()` lê a tecla. O código então traduz os bytes das setas do teclado para as direções 'w', 'a', 's', 'd' e armazena na variável `self.proxima_direcao`.

```
def ler_input(self):  
    if msvcrt.kbhit(): # Verifica sem bloquear  
        key = msvcrt.getch()  
        if key == b'\xe0': # Tecla especial (seta)  
            sub_key = msvcrt.getch()  
            if sub_key == b'H': self.proxima_direcao = 'w'  
            elif sub_key == b'P': self.proxima_direcao = 's'  
            # ... etc ...
```



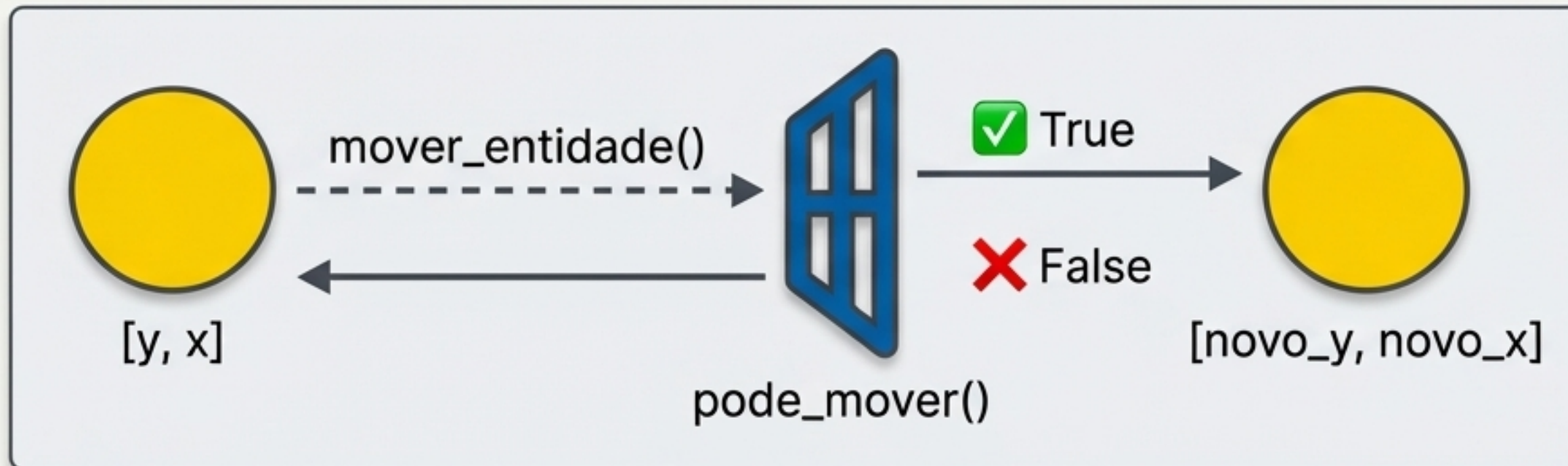
As Regras do Jogo: A Lógica Universal de Movimento e Colisão

A beleza deste código está em sua lógica de movimento reutilizável. Duas funções trabalham juntas:

1. ``pode_mover(y, x)``: A 'verificação de parede'. Ela simplesmente checa se a coordenada ``(y, x)`` existe no conjunto ``caminhos_permitidos`` que criamos na inicialização. É uma operação extremamente rápida.
2. ``mover_entidade(pos, dir)``: Calcula a posição futura com base na direção e usa ``pode_mover`` para validar. Se o movimento for válido, retorna a nova posição; senão, retorna a posição original.

```
def pode_mover(self, y, x):  
    return (y, x) in self.caminhos_permitidos
```

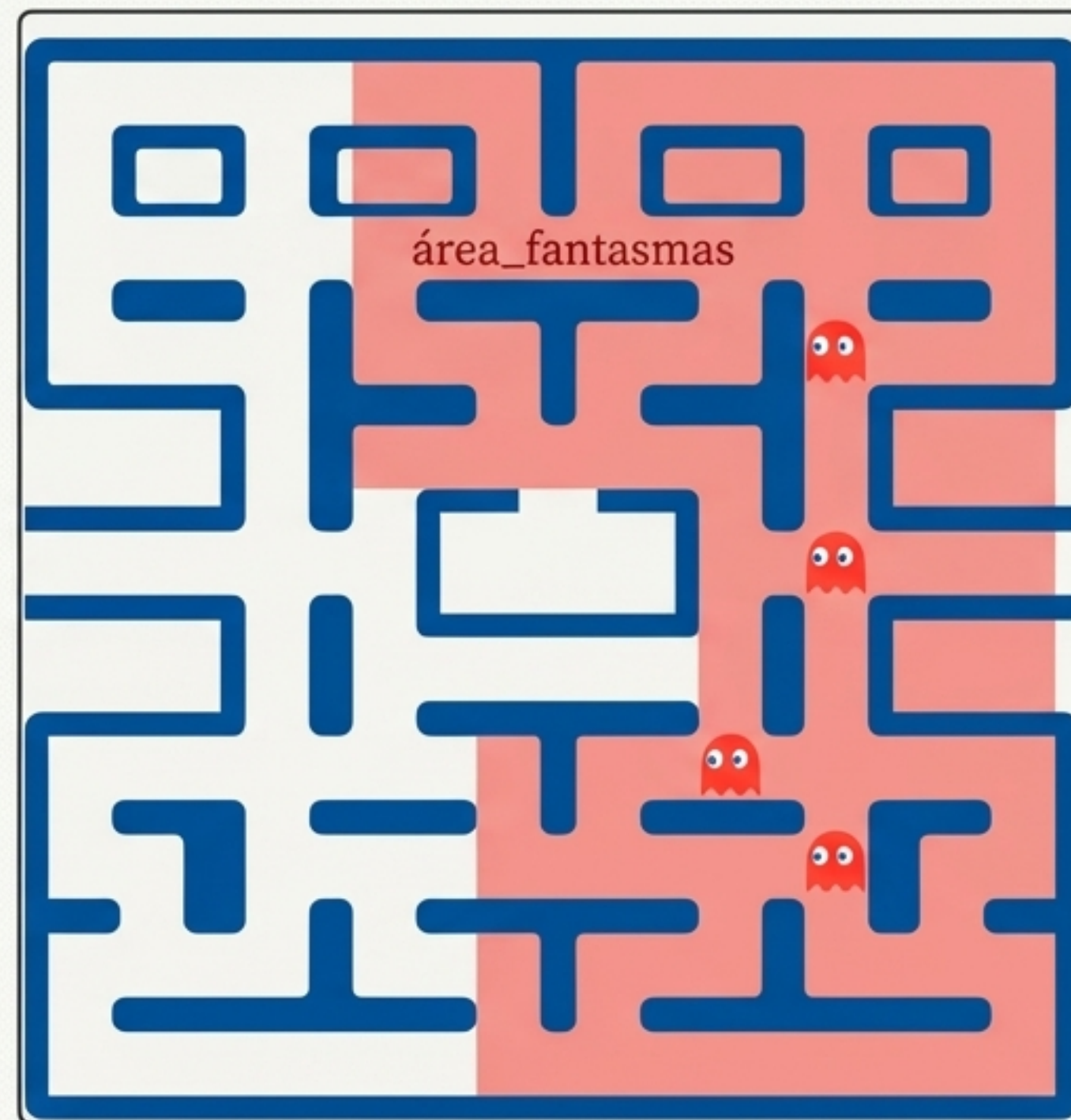
```
def mover_entidade(self, pos_atual, direcao):  
    # ... lógica para calcular novo_y, novo_x ...  
    if self.pode_mover(novo_y, novo_x):  
        return [novo_y, novo_x], True  
    return [y, x], False
```



Os Adversários: A Chegada dos Fantasmas

Os fantasmas são criados durante a inicialização do jogo. Para evitar que comecem muito perto do Pac-Man, o código seleciona um subconjunto de posições válidas (`area_fantasmas`). Em seguida, um loop cria três fantasmas, cada um sendo um dicionário com sua posição (`'pos'`) e direção (`'dir'`), e os posiciona aleatoriamente nessa área.

```
# Dentro de __init__
self.fantasmas = []
if len(posicoes_iniciais_validas) > 10:
    area_fantasmas = posicoes_iniciais_validas[10:]
    for _ in range(3):
        pos = random.choice(area_fantasmas)
        self.fantasmas.append({
            'pos': list(pos),
            'dir': 'd'
        })
```



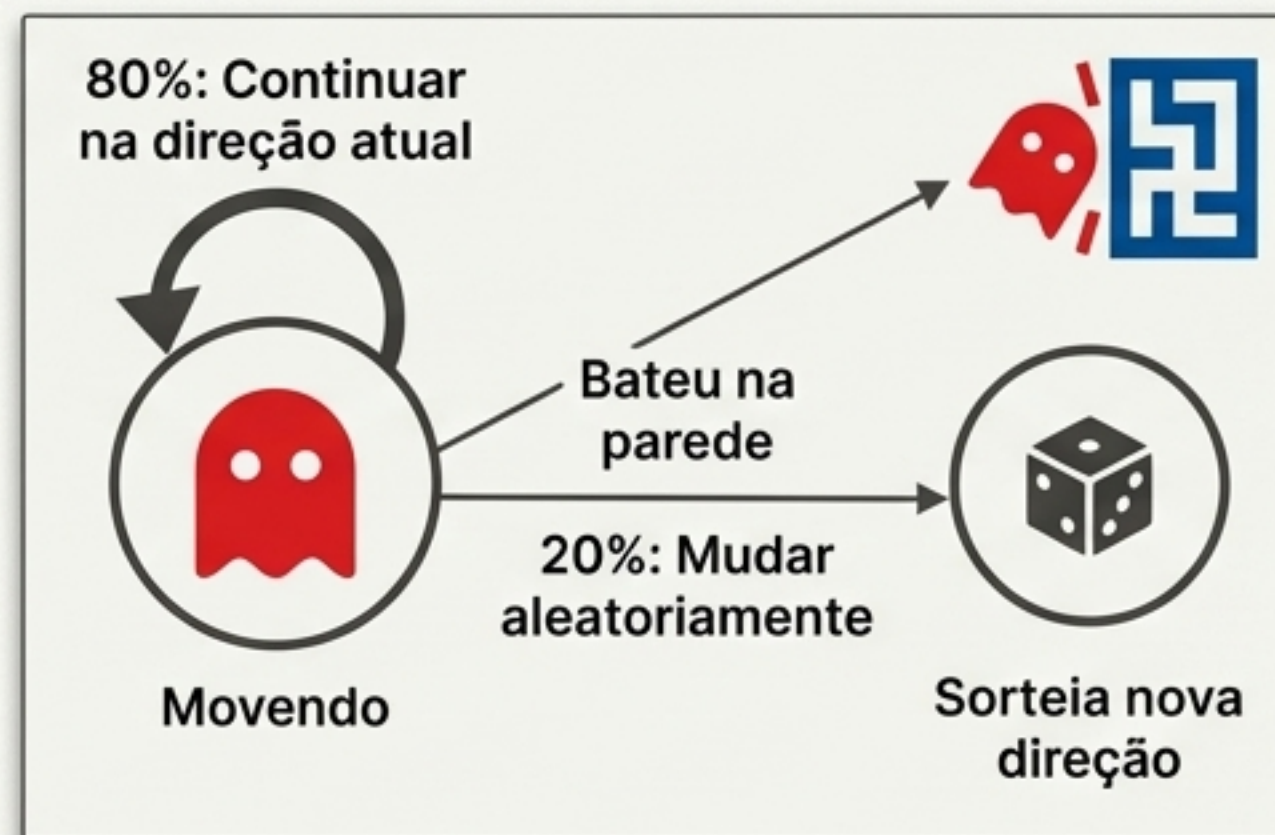
A Caçada: A 'Inteligência' Simples e Eficaz dos Fantasmas

A função `atualizar\_fantasmas` dita o comportamento dos inimigos. A lógica é deliberadamente simples:

- Eles tentam continuar se movendo na direção atual.
- Se baterem em uma parede, são forçados a escolher uma nova direção aleatória.
- Mesmo que não batam, há uma pequena chance (20%) de mudarem de direção aleatoriamente. Isso os impede de ficarem presos em loops e os torna menos previsíveis.
- Após cada movimento, o código verifica se a posição de um fantasma coincide com a do Pac-Man, o que resulta em `game\_over`.

```
# Dentro de atualizar_fantasmas
if moveu:
    f['pos'] = nova_pos
    if random.random() < 0.2: # 20% de chance de mudar
        f['dir'] = random.choice(direcoes)
else: # Bateu na parede
    f['dir'] = random.choice(direcoes)

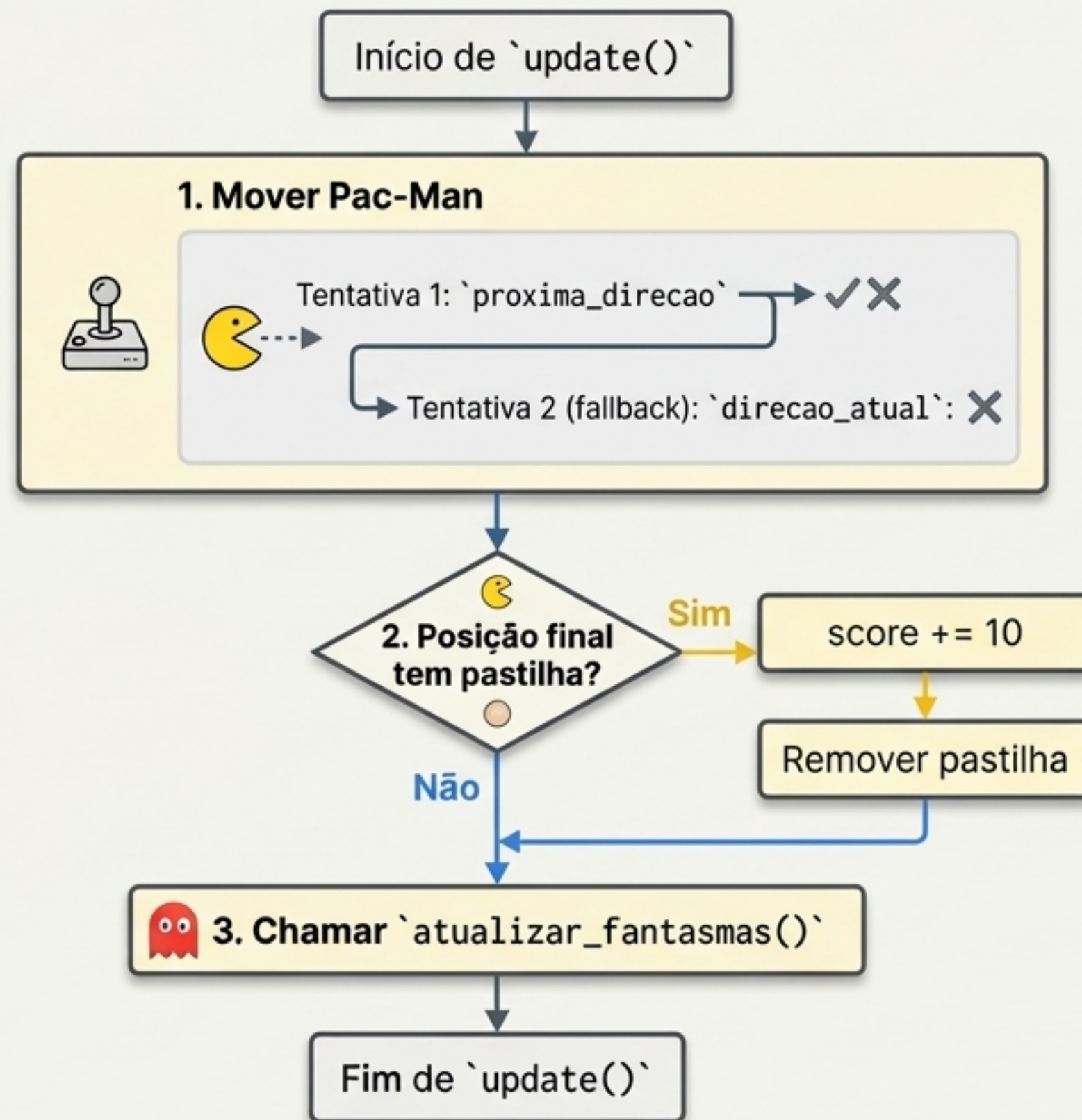
if f['pos'] == self.pacman_pos:
    self.game_over = True
```



O Coração do Jogo: O Ciclo de Vida de um Quadro em `update()`

A função `update` é executada repetidamente e é responsável por avançar o estado do jogo. A cada chamada, ela executa uma sequência lógica precisa de eventos:

- 1. Movimento do Jogador:** Tenta mover o Pac-Man na direção desejada (`proxima\_direcao`). Se não for possível, tenta continuar na direção atual (`direcao\_atual`). Isso cria um controle mais fluido.
- 2. Coleta e Pontuação:** Verifica se a nova posição do Pac-Man está sobre uma pastilha. Se sim, remove a pastilha do mapa e aumenta o `score`.
- 3. Atualização dos Inimigos:** Chama a função `atualizar\_fantasmas()` para que eles também se movam.



A Tela: Traduzindo o Estado do Jogo em Arte no Terminal

A função `desenhar` é a pintora do nosso jogo. Ela constrói a imagem quadro a quadro. Primeiro, ela imprime o código de escape `\033[H` para resetar o cursor para o canto superior esquerdo, criando uma animação fluida em vez de um log que rola. Em seguida, ela itera por cada célula do mapa e desenha em camadas: primeiro o mapa base (paredes), depois as pastilhas, depois os fantasmas (em vermelho) e, por último, o Pac-Man (em amarelo). Isso garante que os personagens sempre apareçam por cima do cenário.

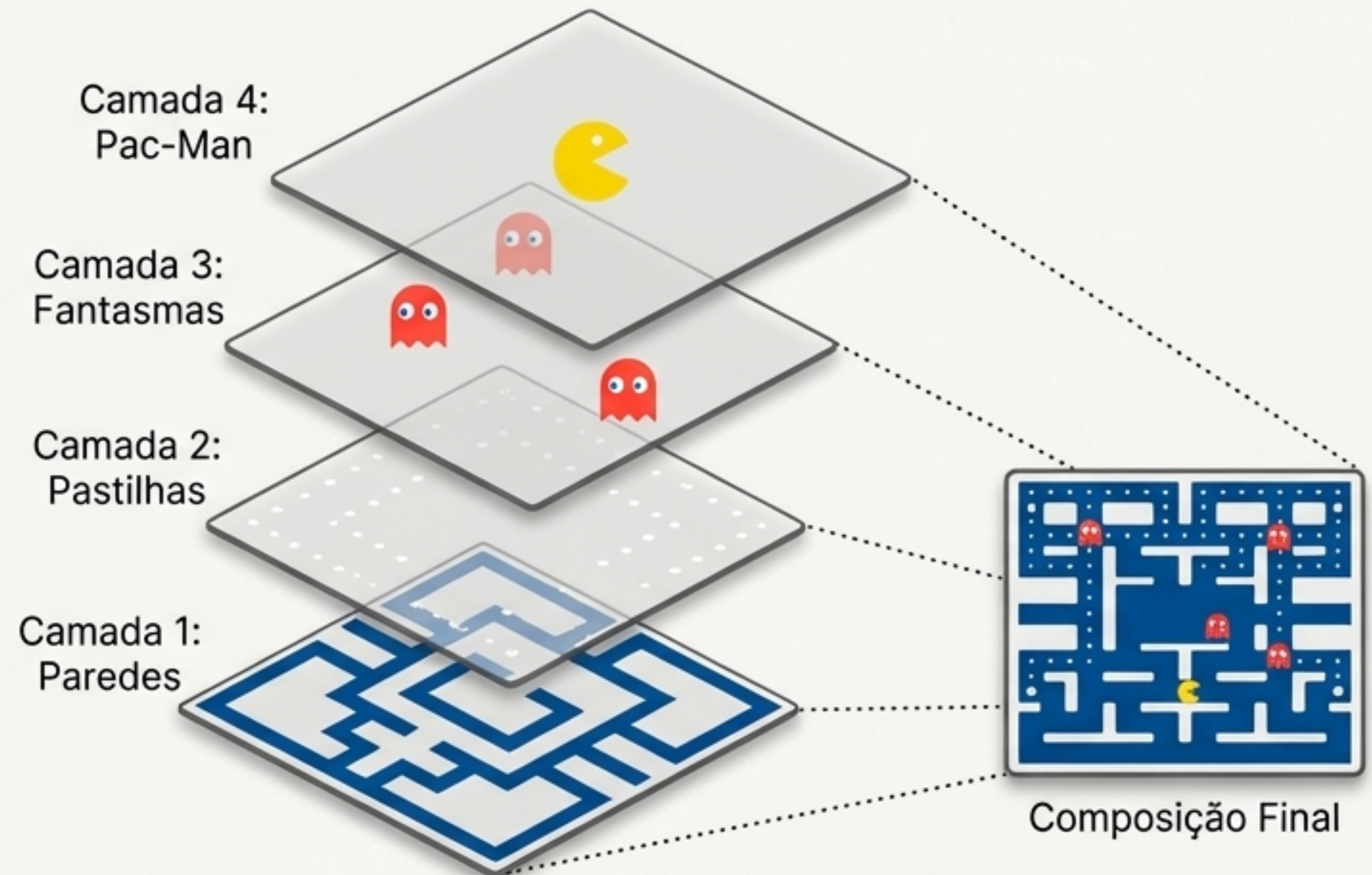
```
# ... loop por y e x ...
char = self.mapa[y][x]
cor = COR_AZUL # Padrão: Parede

if char == PASTILHA:
    cor = COR_RESET

for f in self.fantasmas:
    if f['pos'] == [y, x]:
        char = FANTASMA_SPRITE
        cor = COR_VERMELHO

if self.pacman_pos == [y, x]:
    char = PACMAN_SPRITES[self.direcao_atual]
    cor = COR_AMARELO

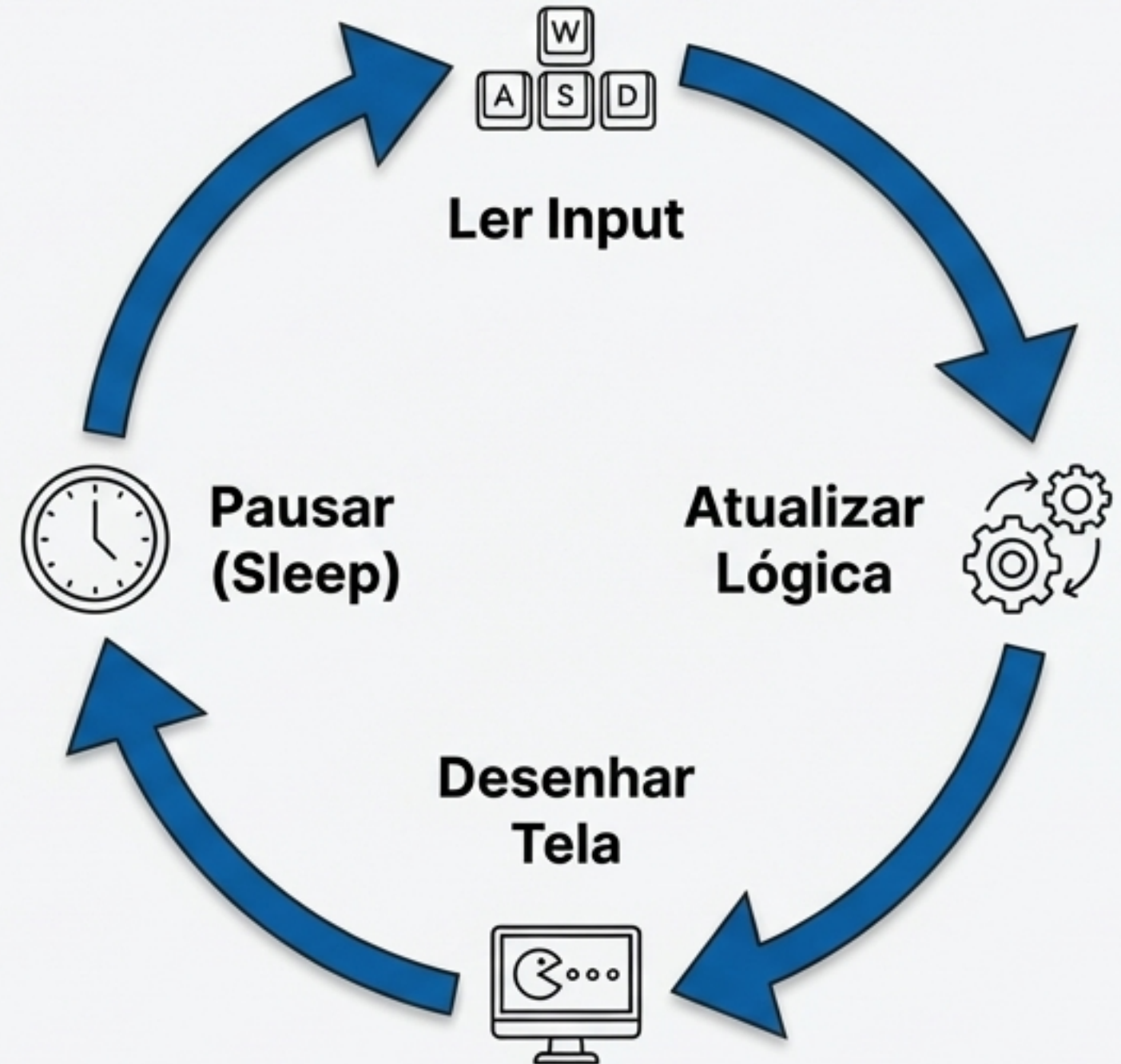
# ... imprime char com cor ...
```



Luzes, Câmera, Ação: Orquestrando o Espetáculo em `main()`

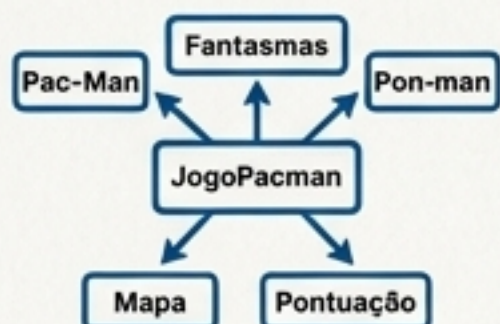
A função `main` é a maestra. Ela inicializa o objeto `JogoPacman` e então entra no `while not jogo.game\_over`, o loop principal do jogo. Dentro deste loop, a sequência é sempre a mesma: ler input, atualizar o estado e desenhar na tela. A chamada a `time.sleep(0.1)` é crucial: ela controla a 'velocidade' do jogo, garantindo uma taxa de quadros (framerate) consistente e jogável.

```
def main():  
    jogo = JogoPacman(MAPA_ORIGINAL)  
    while not jogo.game_over:  
        jogo.ler_input()  
        jogo.update()  
        jogo.desenhar()  
        time.sleep(0.1) # Controle de velocidade
```

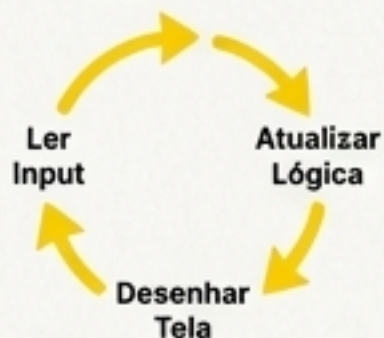


O Jogo é Seu: Conceitos Essenciais e Próximos Desafios

Pilares do Código



Estado Centralizado: Uma única classe (`JogoPacman`) gerencia todas as informações do jogo.



Game Loop Imutável: O ciclo `Input -> Update -> Render` é a espinha dorsal de praticamente qualquer jogo interativo.



Lógica Modular: Funções com responsabilidades claras (`mover_entidade`, `atualizar_fantasma`) tornam o código mais legível e fácil de modificar.

Desafios para Você Expandir o Projeto



IA Aprimorada: Modifique `atualizar_fantasma` para que eles persigam o Pac-Man ativamente em vez de se moverem aleatoriamente.



Power-Ups: Adicione 'Power Pellets' ao mapa que, ao serem comidas, permitam ao Pac-Man caçar os fantasmas por um tempo.



Múltiplos Níveis: Transforme `MAPA_ORIGINAL` em uma lista de strings e adicione a lógica para avançar de nível quando todas as pastilhas forem comidas.